MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Manoj Mishra

Subject & Subject Code: Software Engineering 3CS4-05

Branch: CSE

Batch: A & B

Semester: 3

Session: 2022-23(odd Sem.)

Unit No.: 4

Date of Submission: 1-11-2023

Lect.No.	Topics Covered	Page No.	Total pages
1	Software design Introduction	1-2	2
2	Software design fundamentals	3-5	3
3	Design Approaches	6-8	3
4	Effective modular design	9-12	4
5	Data Architecture design	13-13	1
6	Procedural design	14-15	2
7	Object oriented Design	16-17	1
8	Design documents	18-21	4

References Book:

1. Software engineering 3rd edition K.K Aggarwal
2. Fundamental of software Engineering,pearson by Rajib Mall,

FACULTY HOD DEAN ACADEMICS

2/11/23

Scanned by Scanner Go

UNIT-4 (SOFTWARE DESIGN)

LECTURE NO:-1

Design Fundamentals

Software Design Fundamentals are given following:

1. The Software Design Process

1.1 Definitions and Aspects

Software design is the technical kernel of the software engineering process.

Software design is a process through which the requirements are translated into a representation of software. Initially the representation depicts a holistic view of software. Subsequent refinement leads to a representation that is close to source code.

The software design process has technical and management aspects:

a) from a project management point of view, software design is conducted in two steps:

- preliminary design: concerns the transformation of requirements into data and software architecture;

- detailed design: focuses on refinements to the architectural representation that lead to detailed data structure and algorithmic representations of software;

b) from a technical point of view, software design involves the following steps:

- data design: transforms the information domain model created during the analysis into interaction.

data structures that will be required to implement the software;

- architectural design: defines the relationship among major structural components of the program;

- procedural design: transforms the structural components into a procedural description of the software;

- interface design: establishes the layout and interaction mechanisms for human-machine

MTRC

1.2 Fundamental Concepts

The software designer raises the following questions:

- what criteria can be used to partition the software into individual components?

- how is function or data structure detail separated from a conceptual representation of the software?

- are there uniform criteria that define technical quality of software design?

The basic concepts of the software design process are:

- abstraction

- refinements

- modularity

- architecture

- hierarchy

- data structure

- software procedures

- information hiding

2. Object-Oriented vs. Procedural Design

2.1. Procedural Design

The procedural design is often understood as a software design process that uses mainly control commands such as: sequence, condition, repetition, which are applied to the predefineddata.

Sequences serve to achieve the processing steps in order that is essential in the specification of any algorithm.

Conditions provide facilities for achieving selected processing according to some logical statement.

Repetitions serve to achieve loopings during the computation process.

These three commands are implemented as ready programming language constructs.

MITRC

The programming languages that provide such command constructs are called imperative programming languages.

The software design technique that relies on these constructs is called procedural design, or also structured design.

2.2. Object-Oriented Design

2.2.1. The Macro Design Process

The macro design process is closely related to the **traditional waterfall life cycle**, and serves as controlling framework for the micro process.

The basic philosophy of the macro process is that of incremental development.

The macro design process tends to track the following activities:

Conceptualization: includes establishing the core requirements to the software.

The conceptualization may consist of the following activities:

- assemble an appropriate team to develop a prototype;

- develop a prototype;

- evaluate the prototype.

The prototypes constructed at this stage are meant to be thrown away. Such prototypes are not to be allowed to evolve directly into a system. At this stage any programming paradigm may be used.

Analysis: the purpose of the analysis is to provide a description of a problem.

The description must be complete, consistent, readable, and reviewable. The primary activities during the analysis are:

- identify all primary function points of the system, if possible group them

into clusters of functionally related behaviours; Function points are the outwardly observable and testable behaviours of the system. From the perspective of the user, a function point represents some primary activity of the system in response to some event.

- for each set of function points storyboard a scenario using use-cases;

- develop the use-case view of the system.

During the analysis we seek to model the world by classes from the vocabulary of the problem domain.

Design: the purpose of the design is to create the architecture of the system The primary activities during the design are:

- allocate the clusters of function points to layers and partitions of the architecture;

- perform one spin of corresponding micro design subprocesses !

- validate the architecture by creating an executable release;

- asses the risk of each architectural component.

After the global system construction, release planning sets. A typical order of events for this activity is as follows:

- given the architectural components organize them in order of behaviors;

- allocate the relevant function points to series of architectural releases; Elaborate the Component Diagram and develop the Component view of the system;

- develop the Deployment Diagram and the Deployment view of the system.

- perform task planning, when the work breakdown structure is ready. A product of release planning is a formal development plan, which identifies the stream of architectural releases, team tasks, and risk assessments.

Evolution: this is essentially the process of implementing the system

The primary activities during the evolution process are:

- assign tasks to the team to carry out this release;

- initiate one spin of corresponding micro design subprocesses !

- supervise the micro process by establishing appropriate reviews;

- force closure of the micro process by making an executable release.

In practice the following kinds of changes are to be expected during the system evolution:

- adding new classes to the system;

- changing the implementation of a class;

- changing the representation of a class;

- changing the interface of a class.

Maintenance: this is the activity of managing postdelivery evolution of the system.

The primary activities during the maintenance are:

- prioritize the requests for major enhancement or bug reports that denote systematic problems, and asses the cost of development;

- establish a meaningful collection of these changes and treat them as function points for next elaboration;

- if resources allow it, add more localized enhancements to the next release;

- manage the next evolutionary release.

2.2.2 The Micro Design Process

The micro design process is closely related to the **spiral model** of software development, and serves as framework for iterative and incremental approach to development.

Identifying Classes and Objects: the purpose is to establish the boundaries of the problem.

The activities at this process are:

- apply the classical approach to object-oriented development to generate a set of candidate classes and objects. The abstractions should be directly related to the function points.

- develop Class and Object Diagrams ! For each abstraction do the following:

MTRC

- enumerate its roles and responsibilities;

- devise a sufficient set of operations that satisfy these responsibilities;

- consider each operation in tern and ensure that it is primitive. If not, isolate and expose its more primitive operations.

- consider the needs for construction, destruction and copying;

- consider the need for completeness: add other primitive operations that are not necessarily required by the immediate clients, and would probably be used by future clients.

Identifying the Semantics of Classes and Objects: the purpose is to establish the behavior and attributes of each abstraction. Here we refine the abstractions through distribution of the responsibilities.

A typical order of events might be the following:

- create a scenario or scenarios related to a single function point

- walk through the activities of the scenarios, assigning responsibilities to each abstraction sufficient to accomplish the desired behavior. If necessary, assign attributes that represent structural elements required to carry out certain responsibilities.

- reallocate responsibilities so that there is a reasonably balanced distribution of behavior. Where possible reuse or adapt existing responsibilities. Splitting large responsibilities to smaller ones is very common action.

- elaborate the State Diagrams and Sequence Diagrams.

Identifying the Relationships among Classes and Objects: the purpose is to solidify the boundaries of and to recognize the collaborations with each abstraction. This activity formalizes the conceptual as well as physical separations of concern among the abstractions.

The activities night be:

- collect a set of classes at a given level of abstraction and add the significant properties relevant to the problem being modeled;

MTRC

- establish association relationships in case of presented semantic dependencies; Some associations may immediately be identified as specialization/generalization or aggregation relationships;

- for each association specify the role of each participant, as well as any relevant cardinality or other kind of constraint;

- validate the decisions by walk through scenarios;

- develop Collaboration Diagrams.

Inheritance, containment, instantiation and use are the main kinds of relationships of interest, together with other properties as labels, cardinality, and so on:

- given a collection of classes clustered by some set of associations, look for patterns of behavior that represent opportunities for specialization/generalization. Place the classes in context of the inheritance lattice, or fabricate another one.

- if there are patterns of structure, consider creating new classes that capture this common structure, and introduce them either through inheritance or through aggregation;

- as development proceeds, introduce tactical details such as: roles, keys, cardinalities, friendship, and so on.

Implementing Classes and Objects: the purpose is to provide a refinement of the existing abstractions, sufficient to unveil new classes and objects at the next level of abstraction, which we feed at the next iteration. A typical order of events may be the following:

- for each class consider its protocol. Determine which operations need to be optimized. Develop precise signatures for all significant operations;

- adjust the inheritance lattice if required;

- alter local implementations of abstractions that are not time or space efficient;

- check whether the algorithms for the operations are suitable.

3. Design Documentation:

A **software design description** (a.k.a. **software design document** or **SDD**; just **design document**; also Software Design Specification) is a written description of a software product, that a software designer writes in order to give a software development team overall guidance to the architecture of the software project. An SDD usually accompanies an architecture diagram with pointers to detailed feature specifications of smaller pieces of the design. Practically, the description is required to coordinate a large team under a single vision, needs to be a stable reference, and outline all parts of the software and how they will work.

The SDD usually contains the following information:

1.The data design describes structures that reside within the software. Attributes and relationships between data objects dictate the choice of data structures.
2.The architecture design uses information flowing characteristics, and maps them into the program structure. The transformation mapping method is applied to exhibit distinct boundaries between incoming and outgoing data. The data flow diagrams allocate control input, processing and output along three separate modules.
3.The interface design describes internal and external program interfaces, as well as the design of the human interface. Internal and external interface designs are based on the information obtained from the analysis model.
4.The procedural design describes structured programming concepts using graphical, tabular and textual notations.
These design mediums enable the designer to represent procedural detail, that facilitates translation to code. This blueprint for implementation forms the basis for all subsequent software engineering work.

MTRC

LECTURE NO:-2

Effective Moduler Design

The role of effective modular design in software engineering: Any software comprises of many systems which contains several sub-systems and those sub-systems further contains their sub-systems. So, designing a complete system in one go comprising of each and every required functionality is a hectic work and the process can have many errors because of its vast size.

Thus in order to solve this problem the developing team breakdown the complete software into various modules. A module is defined as the unique and addressable components of the software which can be solved and modified independently without disturbing (or affecting in very small amount) other modules of the software. Thus every software design should follow modularity.

The process of breaking down a software into multiple independent modules where each module is developed separately is called Modularization.

Effective modular design can be achieved if the partitioned modules are separately solvable, modifiable as well as compilable. Here separate compilable modules means that after making changes in a module there is no need of recompiling the whole software system.

In order to build a software with effective modular design there is a factor "Functional Independence" which comes into play. The meaning of Functional Independence is that a function is atomic in nature so that it performs only a single task of the software without or with least interaction with other modules. Functional Independence is considered as a sign of growth in modularity i.e., presence of larger functional independence results in a software system of good design and design further affects the quality of the software.

Benefits of Independent modules/functions in a software design: Since the functionality of the software have been broken down into atomic levels, thus developers get a clear requirement of each and every functions and hence designing of the software becomes easy and error free.

Notes By: – Shashi Kant(Asst. Prof, CSE Department)

As the modules are independent they have limited or almost no dependency on other modules. So, making changes in a module without affecting the whole system is possible in this approach. Error propagation from one module to another and further in whole system can be neglected and it saves time during testing and debugging.

Moreover, Modules should be specified and designed so that the internal details of modules should be invisible or inaccessible to other modules. Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function. Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module.

Major benefits: reduce the change impacts in testing and maintenance.

Functional independence:

Functional independence is achieved by developing modules with "single minded" function and an "aversion" to excessive interaction with other modules.
Independent modules are easier to maintain because secondary effects caused by design or code modification are limited, error propagation is reduced and reusable modules are possible.

Design modules based on independent functional features
Major benefits: effective modularity

Independence of modules of a software system can be measured using 2 criteria : **Cohesion, and Coupling.** These are explained as following below.

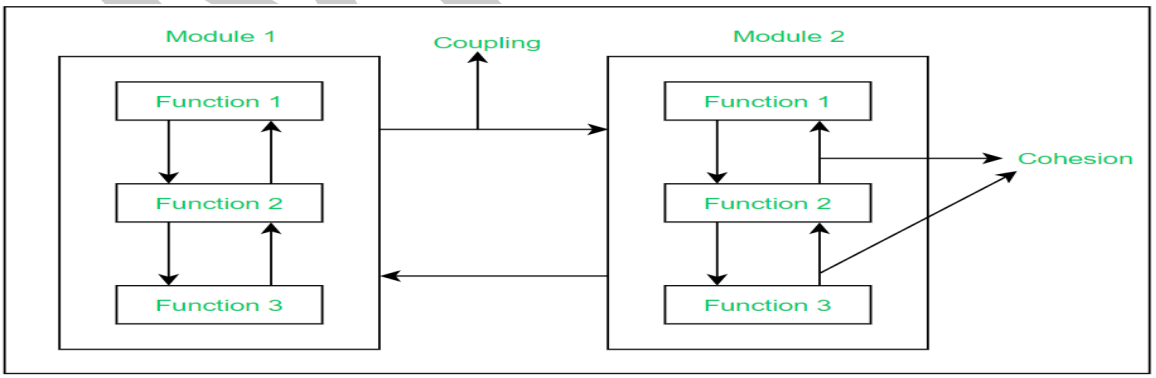


Figure – Cohesion and Coupling between 2 modules

Cohesion:

Cohesion is a measure of strength in relationship between various functions within a module.

OR

a natural extension of the information hiding concept a module may perform a number of tasks. A cohesive module performs a single task in a procedure with little interactions with others.

It is of 7 types which are listed below in the order of high to low cohesion:

1. Functional cohesion
2. Sequential cohesion
3. Communicational cohesion
4. Procedural cohesion
5. Temporal cohesion
6. Logical cohesion
7. Co-incidental cohesion

Coupling:

Coupling is a measure of strength in relationship between various modules within a software.

OR

A measure of interconnection among modules in a program structure. Coupling depends on the interface complexity between modules.

Goal: to strive for lowest possible coupling among modules. Good coupling —> reduce or avoid change impact and ripple effects. —> reduce the cost in program changes, testing, maintenance

Types of coupling:

It is of 6 types which are listed below in the order of low to high coupling:

1. data coupling: parameter passing or data interaction
2. control coupling: share related control logical (for a control data)
3. common coupling: common data sharing
4. content coupling: module A use of data or control information maintained in another module.

5. stamp coupling

6. external coupling

Note: A good software design requires **high cohesion** and **low coupling**.

MITRC

LECTURE NO:-3

Design Documentation

A **software design description** (a.k.a. **software design document** or **SDD**; just **design document**; also Software Design Specification) is a written description of a software product, that a software designer writes in order to give a software development team overall guidance to the architecture of the software project. An SDD usually accompanies an architecture diagram with pointers to detailed feature specifications of smaller pieces of the design. Practically, the description is required to coordinate a large team under a single vision, needs to be a stable reference, and outline all parts of the software and how they will work.

The SDD usually contains the following information:

1. The data design describes structures that reside within the software. Attributes and relationships between data objects dictate the choice of data structures.
2. The architecture design uses information flowing characteristics, and maps them into the program structure. The transformation mapping method is applied to exhibit distinct boundaries between incoming and outgoing data. The data flow diagrams allocate control input, processing and output along three separate modules.
3. The interface design describes internal and external program interfaces, as well as the design of the human interface. Internal and external interface designs are based on the information obtained from the analysis model.
5. The procedural design describes structured programming concepts using graphical, tabular and textual notations.

These design mediums enable the designer to represent procedural detail, that facilitates translation to code. This blueprint for implementation forms the basis for all subsequent software engineering work.

Notes By: – Shashi Kant(Asst. Prof, CSE Department)

LECTURE NO:-4

Procedural Design

Software design is a process to conceptualize the software requirements into software implementation. Software design takes the user requirements as challenges and tries to find optimum solution. While the software is being conceptualized, a plan is chalked out to find the best possible design for implementing the intended solution.

There are multiple variants of software design. Let us study them briefly:

Structured Design

Structured design is a conceptualization of problem into several well-organized elements of solution. It is basically concerned with the solution design. Benefit of structured design is, it gives better understanding of how the problem is being solved. Structured design also makes it simpler for designer to concentrate on the problem more accurately.

Structured design is mostly based on 'divide and conquer' strategy where a problem is broken into several small problems and each small problem is individually solved until the whole problem is solved.

The small pieces of problem are solved by means of solution modules. Structured design emphasis that these modules be well organized in order to achieve precise solution.

These modules are arranged in hierarchy. They communicate with each other. A good structured design always follows some rules for communication among multiple modules, namely -

Cohesion - grouping of all functionally related elements.

Coupling - communication between different modules.

A good structured design has high cohesion and low coupling arrangements.

Notes By: – Shashi Kant(Asst. Prof, CSE Department)

Function Oriented Design

In function-oriented design, the system is comprised of many smaller sub-systems known as functions. These functions are capable of performing significant task in the system. The system is considered as top view of all functions.

Function oriented design inherits some properties of structured design where divide and conquer methodology is used.

This design mechanism divides the whole system into smaller functions, which provides means of abstraction by concealing the information and their operation.. These functional modules can share information among themselves by means of information passing and using information available globally.

Another characteristic of functions is that when a program calls a function, the function changes the state of the program, which sometimes is not acceptable by other modules. Function oriented design works well where the system state does not matter and program/functions work on input rather than on a state.

Design Process

- The whole system is seen as how data flows in the system by means of data flow diagram.
- DFD depicts how functions changes data and state of entire system.
- The entire system is logically broken down into smaller units known as functions on the basis of their operation in the system.
- Each function is then described at large.

Object Oriented Design

Object oriented design works around the entities and their characteristics instead of functions involved in the software system. This design strategies focuses on entities and its characteristics. The whole concept of software solution revolves around the engaged entities.

Let us see the important concepts of Object Oriented Design:

- **Objects** - All entities involved in the solution design are known as objects. For example, person, banks, company and customers are treated as objects.

Every entity has some attributes associated to it and has some methods to perform on the attributes.

•**Classes** - A class is a generalized description of an object. An object is an instance of a class. Class defines all the attributes, which an object can have and methods, which defines the functionality of the object.

In the solution design, attributes are stored as variables and functionalities are defined by means of methods or procedures.

•**Encapsulation** - In OOD, the attributes (data variables) and methods (operation on the data) are bundled together is called encapsulation. Encapsulation not only bundles important information of an object together, but also restricts access of the data and methods from the outside world. This is called information hiding.

•**Inheritance** - OOD allows similar classes to stack up in hierarchical manner where the lower or sub-classes can import, implement and re-use allowed variables and methods from their immediate super classes. This property of OOD is known as inheritance. This makes it easier to define specific class and to create generalized classes from specific ones.

•**Polymorphism** - OOD languages provide a mechanism where methods performing similar tasks but vary in arguments, can be assigned same name. This is called polymorphism, which allows a single interface performing tasks for different types. Depending upon how the function is invoked, respective portion of the code gets executed.

Design Process

Software design process can be perceived as series of well-defined steps. Though it varies according to design approach (function oriented or object oriented, yet It may have the following steps involved:

•A solution design is created from requirement or previous used system and/ or system sequence diagram.

•Objects are identified and grouped into classes on behalf of similarity in attribute characteristics.

•Class hierarchy and relation among them is defined.

•Application framework is defined.

Software Design Approaches

Here are two generic approaches for software designing:

Top Down Design

We know that a system is composed of more than one sub-systems and it contains a number of components. Further, these sub-systems and components may have their on set of sub-system and components and creates hierarchical structure in the system.

Top-down design takes the whole software system as one entity and then decomposes it to achieve more than one sub-system or component based on some characteristics. Each sub-system or component is then treated as a system and decomposed further. This process keeps on running until the lowest level of system in the top-down hierarchy is achieved.

Top-down design starts with a generalized model of system and keeps on defining the more specific part of it. When all components are composed the whole system comes into existence.

Top-down design is more suitable when the software solution needs to be designed from scratch and specific details are unknown.

Bottom-up Design

The bottom up design model starts with most specific and basic components. It proceeds with composing higher level of components by using basic or lower level components. It keeps creating higher level components until the desired system is not evolved as one single component. With each higher level, the amount of abstraction is increased.

Bottom-up strategy is more suitable when a system needs to be created from some existing system, where the basic primitives can be used in the newer system.

Both, top-down and bottom-up approaches are not practical individually. Instead, a good combination of both is used.

LECTURE NO:-5

Architectural Design

Introduction: The software needs the architectural design to represents the design of software. IEEE defines architectural design as "the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system." The software that is built for computer-based systems can exhibit one of these many architectural styles.

Each style will describe a system category that consists of :

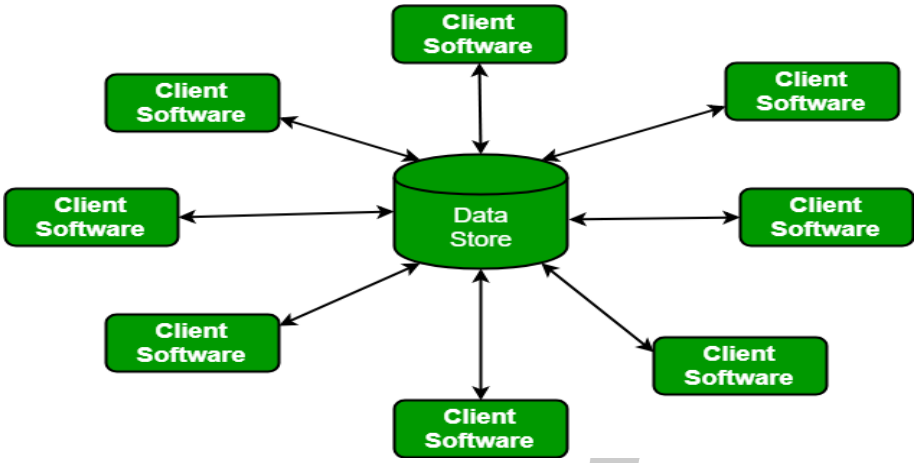
- A set of components(eg: a database, computational modules) that will perform a function required by the system.
- The set of connectors will help in coordination, communication, and cooperation between the components.
- Conditions that how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

The use of architectural styles is to establish a structure for all the components of the system.

Taxonomy of Architectural styles:

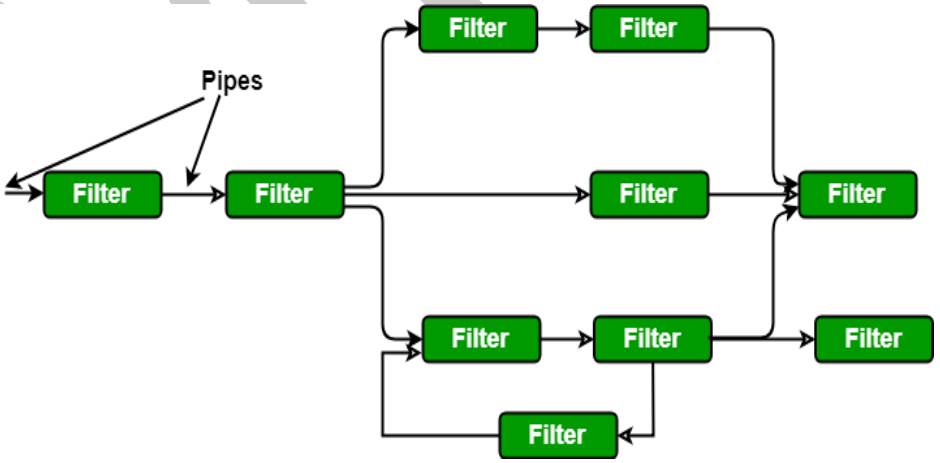
1. Data centred architectures:
    - A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.
    - The figure illustrates a typical data centered style. The client software access a central repository. Variation of this approach are used to transform the repository into a blackboard when data related to client or data of interest for the client change the notifications to client software.
    - This data-centered architecture will promote integrability. This means that the existing components can be changed and new client components can be added to the architecture without the permission or concern of other clients.
    - Data can be passed among clients using blackboard mechanism.

Notes By: – Shashi Kant(Asst. Prof, CSE Department)

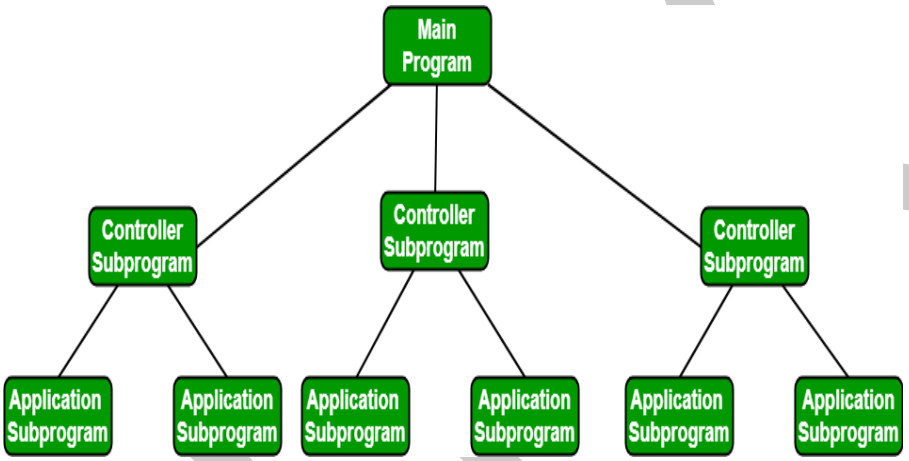


2. Data flow architectures:

- This kind of architecture is used when input data to be transformed into output data through a series of computational manipulative components.
- The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by pipes.
- Pipes are used to transmit data from one component to the next.
- Each filter will work independently and is designed to take data input of a certain form and produces data output to the next filter of a specified form. The filters don't require any knowledge of the working of neighboring filters.
- If the data flow degenerates into a single line of transforms, then it is termed as batch sequential. This structure accepts the batch of data and then applies a series of sequential components to transform it.



3. Call and Return architectures: It is used to create a program that is easy to scale and modify. Many sub-styles exist within this category. Two of them are explained below.
    - Remote procedure call architecture: This components is used to present in a main program or sub program architecture distributed among multiple computers on a network.
    - Main program or Subprogram architectures: The main program structure decomposes into number of subprograms or function into a control hierarchy. Main program contains number of subprograms that can invoke other components.



4. Object Oriented architecture: The components of a system encapsulate data and the operations that must be applied to manipulate the data. The coordination and communication between the components are established via the message passing.

5. Layered architecture:
    - A number of different layers are defined with each layer performing a well-defined set of operations. Each layer will do some operations that becomes closer to machine instruction set progressively.
    - At the outer layer, components will receive the user interface operations and at the inner layers, components will perform the operating system interfacing(communication and coordination with OS)